01_mock: Tool-Using Support Agent

Timebox: **55 minutes**
Language: **Python (Colab)**

This is the highest-priority mock for your interview shape.

Scenario

Implement an agent loop that uses local tools to inspect orders, evaluate policy, and submit refunds.

What to implement

1. `build_agent_system_prompt`
2. `build_tool_schemas`
3. `validate_tool_call`
4. `execute_tool_call`
5. `run_agent`

Completion criteria (required)

- Prompting: system prompt clearly drives tool-grounded behavior and clarification behavior
- Tool schemas are built in Claude-style (`name`, `description`, `input_schema`) and passed to model calls
- Correct Claude Messages API loop (`assistant:tool_use` -> `user:tool_result` -> continue)
- Multiple tool calls in one response
- Error-safe behavior for invalid args + runtime exceptions
- `max_steps` protection against infinite loops

Time guidance

- 10 min: read scaffold + plan flow
- 35 min: implement required functions
- 10 min: run tests + edge-case cleanup

In [ ]:
```
import inspect
import json
```

```python
from copy import deepcopy
from typing import Any, Callable

ORDERS_DB = {
    "u-100": [
        {"order_id": "o-900", "status": "delivered", "days_since_delivery":
    ],
    "u-200": [
        {"order_id": "o-901", "status": "in_transit", "days_since_delivery":
    ],
}
REFUNDS: list[dict[str, Any]] = []


def reset_state() -> None:
    REFUNDS.clear()


def get_orders(user_id: str) -> list[dict[str, Any]]:
    if user_id == "boom":
        raise RuntimeError("orders backend unavailable")
    return deepcopy(ORDERS_DB.get(user_id, []))


def policy_check(order_id: str, reason: str, days_since_delivery: int) -> di
    eligible = reason.lower() in {"damaged", "wrong_item"} and days_since_de
    return {
        "order_id": order_id,
        "eligible": eligible,
        "policy_reason": "allowed" if eligible else "outside_policy",
    }


def create_refund(order_id: str, amount: float) -> dict[str, Any]:
    if amount <= 0:
        raise ValueError("refund amount must be positive")
    record = {"order_id": order_id, "amount": amount, "status": "submitted"}
    REFUNDS.append(record)
    return deepcopy(record)


TOOL_REGISTRY: dict[str, Callable[..., Any]] = {
    "get_orders": get_orders,
    "policy_check": policy_check,
    "create_refund": create_refund,
}


class ScriptedModel:
    def __init__(self, responses: list[dict[str, Any]]) -> None:
        self._responses = deepcopy(responses)
        self._index = 0
        self.last_system_prompt: str | None = None
        self.last_tools: list[dict[str, Any]] = []

    def __call__(self, messages: list[dict[str, Any]], system_prompt: str, t
```

```
        self.last_system_prompt = system_prompt
        self.last_tools = deepcopy(tools)
        if self._index >= len(self._responses):
            return {
                "stop_reason": "end_turn",
                "content": [{"type": "text", "text": "No scripted response l
            }
        response = self._responses[self._index]
        self._index += 1
        return deepcopy(response)
```

In [ ]:

```
def build_agent_system_prompt() -> str:
    """Return concise system instructions for a safe tool-using support ager
    # TODO:
    # - Tell the model to use tools for factual checks before claiming accou
    # - Tell the model to never fabricate tool outputs.
    # - Tell the model to ask a clarifying question when required policy inp
    # - Keep final answers concise and grounded in tool results.
    raise NotImplementedError


def build_tool_schemas(tool_registry: dict[str, Callable[..., Any]]) -> list
    """Build minimal Claude-style tool definitions from the local tool regis
    # TODO:
    # - Return list entries with keys: name, description, input_schema.
    # - input_schema must be JSON Schema object with required fields from fu
    raise NotImplementedError


def validate_tool_call(tool_call: dict[str, Any], tool_registry: dict[str, C
    """Return an error string if invalid, otherwise None."""
    # TODO:
    # 1) Ensure tool_call has id/name/input.
    # 2) Ensure tool exists in registry.
    # 3) Ensure input is a dict.
    # 4) Ensure all required function args are present.
    raise NotImplementedError


def execute_tool_call(tool_call: dict[str, Any], tool_registry: dict[str, Ca
    """Return Claude-style tool_result content block."""
    # TODO:
    # - Call validate_tool_call first.
    # - Execute valid tools with **tool_call["input"].
    # - Catch runtime exceptions and return is_error=True payload.
    # Return block shape:
    # {
    #   "type": "tool_result",
    #   "tool_use_id": "...",
    #   "is_error": bool,
    #   "content": "json-string"
    # }
    raise NotImplementedError


def run_agent(
```

```
    user_prompt: str,
    model: Callable[[list[dict[str, Any]], str, list[dict[str, Any]]], dict[
    tool_registry: dict[str, Callable[..., Any]],
    max_steps: int = 6,
) -> dict[str, Any]:
    """Run Claude-style tool-use loop until end_turn or max_steps exhaustion
    # TODO:
    # - Initialize messages in Messages API shape.
    # - Build and pass system prompt to model on every model call.
    # - Build tool schemas and pass them to model on every model call.
    # - Loop up to max_steps.
    # - Append assistant response content each turn.
    # - On stop_reason == "tool_use", execute all tool_use blocks and append
    #   with only tool_result blocks (in the same order).
    # - On stop_reason == "end_turn", return {"final_text": ..., "messages":
    # - Raise RuntimeError("max_steps_exceeded") if no end_turn in time.
    raise NotImplementedError
```

Run Tests

Run this final test cell after implementing all TODO sections.

In [ ]:

```
def _tool_result_blocks(messages: list[dict[str, Any]]) -> list[dict[str, Ar
    blocks: list[dict[str, Any]] = []
    for message in messages:
        if message.get("role") != "user":
            continue
        for block in message.get("content", []):
            if block.get("type") == "tool_result":
                blocks.append(block)
    return blocks


def run_exam01_tests() -> None:
    reset_state()
    prompt = build_agent_system_prompt().lower()
    assert "use tools" in prompt
    assert "never fabricate" in prompt
    assert "clarifying question" in prompt
    tool_defs = build_tool_schemas(TOOL_REGISTRY)
    assert sorted(tool["name"] for tool in tool_defs) == sorted(TOOL_REGISTR

    # 1) Single tool call
    model = ScriptedModel(
        [
            {
                "stop_reason": "tool_use",
                "content": [
                    {"type": "tool_use", "id": "t1", "name": "get_orders", "
                ],
            },
            {"stop_reason": "end_turn", "content": [{"type": "text", "text":
        ]
    )
```

```
    result = run_agent("Where is my order?", model, TOOL_REGISTRY)
    assert "delivered" in result["final_text"].lower()
    assert len(_tool_result_blocks(result["messages"])) == 1
    assert model.last_system_prompt is not None
    assert "use tools" in model.last_system_prompt.lower()
    assert sorted(tool["name"] for tool in model.last_tools) == sorted(TOOL_

    # 2) Multiple tools in one model turn
    model = ScriptedModel(
        [
            {
                "stop_reason": "tool_use",
                "content": [
                    {
                        "type": "tool_use",
                        "id": "t2",
                        "name": "policy_check",
                        "input": {"order_id": "o-900", "reason": "damaged",
                    },
                    {
                        "type": "tool_use",
                        "id": "t3",
                        "name": "create_refund",
                        "input": {"order_id": "o-900", "amount": 42.5},
                    },
                ],
            },
            {"stop_reason": "end_turn", "content": [{"type": "text", "text":
        ]
    )
    result = run_agent("Refund my damaged item", model, TOOL_REGISTRY)
    assert len(_tool_result_blocks(result["messages"])) == 2
    assert REFUNDS and REFUNDS[-1]["order_id"] == "o-900"
    assistant_idx = next(i for i, m in enumerate(result["messages"]) if m["r
    assert result["messages"][assistant_idx + 1]["role"] == "user"
    assert all(
        block.get("type") == "tool_result" for block in result["messages"][a
    )

    # 3) Missing args -> is_error tool message
    model = ScriptedModel(
        [
            {
                "stop_reason": "tool_use",
                "content": [{"type": "tool_use", "id": "bad-args", "name": "
            },
            {"stop_reason": "end_turn", "content": [{"type": "text", "text":
        ]
    )
    result = run_agent("debug", model, TOOL_REGISTRY)
    tool_block = _tool_result_blocks(result["messages"])[0]
    assert tool_block["is_error"] is True
    assert "missing_required_args" in tool_block["content"]

    # 4) Runtime exception -> is_error tool message
    model = ScriptedModel(
```

```
        [
            {
                "stop_reason": "tool_use",
                "content": [
                    {"type": "tool_use", "id": "boom", "name": "get_orders",
                ],
            },
            {"stop_reason": "end_turn", "content": [{"type": "text", "text":
        ]
    )
    result = run_agent("debug", model, TOOL_REGISTRY)
    tool_block = _tool_result_blocks(result["messages"])[0]
    assert tool_block["is_error"] is True
    assert "backend unavailable" in tool_block["content"]

    # 5) Max step protection
    model = ScriptedModel(
        [
            {
                "stop_reason": "tool_use",
                "content": [{"type": "tool_use", "id": "loop1", "name": "get
            },
            {
                "stop_reason": "tool_use",
                "content": [{"type": "tool_use", "id": "loop2", "name": "get
            },
            {
                "stop_reason": "tool_use",
                "content": [{"type": "tool_use", "id": "loop3", "name": "get
            },
        ]
    )
    try:
        run_agent("loop", model, TOOL_REGISTRY, max_steps=2)
        raise AssertionError("Expected max_steps_exceeded")
    except RuntimeError as exc:
        assert "max_steps_exceeded" in str(exc)

    print("01_mock tests passed")


run_exam01_tests()
```